CrewAI 最终实战：习题生成与考试评测系统

- 来源: https://class.imooc.com/lesson/5293#mid=105158
- 阶段: 阶段六: 【智能体】从单体到多体
- 周: 第12周 Multi-Agent：多智能体协作
- 模块: CrewAI 实战
- 课节: 图文 1-2 CrewAI 最终实战：习题生成与考试评测系统

1-2 CrewAI 最终实战：习题生成与考试评测系统

CrewAI 最终实战：习题生成与考试评测系统

前面两节我们系统学习了A2A协议和CrewAI框架的理论基础。现在，让我们通过一个完整的实战项目来巩固所学知识：智能练习题系统。

这个系统展示了如何将CrewAI框架应用到真实教育场景，实现了从题目生成、学生答题、AI批改到学习分析的完整闭环。本节不会逐行讲解代码，而是聚焦于系统架构、设计思路和关键技术点，帮助你理解如何将理论转化为实践。

> [!TIP] 完整源码位于：samples/chapter12/exercise_system/ 建议边阅读文档边查看源码，理解系统的设计思想。

系统概览

核心设计理念

这个系统采用了双程序架构，将教师和学生的职责清晰分离。每个程序包含传统业务逻辑和AI智能决策两类任务。

程序1: 题目生成器 (管理员/教师使用)

程序1: 题目生成器

```
question_generator.py (Sequential模式)

──────────────────────────────────────────

  阶段1: 设计题目框架
  • AI决策: 分析大纲, 规划题目结构
  • 输出: 题目设计方案
──────────────────────────────────────────

  阶段2: 生成具体题目
  • AI决策: 生成题目内容和选项
  • 输出: JSON格式题目
──────────────────────────────────────────

  阶段3: 数据持久化
  • 传统业务: 保存到 data/questions.json
──────────────────────────────────────────
```

程序2: 考试系统 (学生使用)

程序2: 考试系统

```
main.py (Hierarchical模式)

──────────────────────────────────────────

  阶段1: 组卷
  • 传统业务: 从题库随机抽题
  • 逻辑: min(题库总数, 配置数量)
──────────────────────────────────────────

  阶段2: 学生答题
  • 传统业务: 收集学生输入（A/B/C/D）
──────────────────────────────────────────

  阶段3: 自动批改
  • 传统业务: 对比答案, 计算分数
  • 无需AI: 选择题答案确定
──────────────────────────────────────────

  阶段4: AI深度分析（可选, 用户询问）
  • AI决策: Manager协调2个Agent
    - Agent1: 答案解析专家
    - Agent2: 学习诊断教练
──────────────────────────────────────────
```

关键设计原则:

关键设计原则

- ✓ 非常重要：并非所有的业务都要用AI！这是新手最常见的误区，AI业务与非AI业务可以搭配。能不用AI就尽量不要用AI，切记，否则你会发现你的token在疯狂燃烧。
- ✓ AI负责创造性任务：生成题目、分析错题、个性化建议

AI负责创造性任务

- ✓ 传统代码负责确定性任务：随机抽题、对比答案、数据存储

传统代码负责确定性任务

目录结构

系统采用模块化设计，每个模块职责清晰：

```
exercise_system/
│
├── core/                          # 核心基础模块
│   ├── __init__.py
│   ├── config.py                # 配置管理（API密钥、模型参数）
│   ├── llm.py                   # LLM初始化
│   ├── database.py              # 题库管理（增删改查）
│   ├── models.py                # 数据模型（Question、Option等）
│   └── tools.py                 # 工具函数
│
├── agents/                        # Agent角色定义
│   ├── __init__.py
│   ├── question_expert.py       # 题目生成专家Agent
│   ├── explanation_agent.py     # 答案解析专家Agent
│   └── learning_advisor.py      # 学习诊断教练Agent
│
├── tasks/                         # Task任务定义
│   ├── __init__.py
│   ├── question_tasks.py        # 题目生成任务
│   ├── explanation_tasks.py     # 答案解析任务
│   └── learning_tasks.py        # 学习建议任务
│
├── crews/                         # Crew团队组织
│   ├── __init__.py
│   └── exercise_crew.py         # 统一的Crew（单一Crew架构）
│
├── ui/                            # 用户界面
│   ├── __init__.py
│   ├── console.py               # Rich控制台配置
│   └── display.py               # 显示函数（欢迎界面、题目展示、结果报告）
│
├── data/                          # 数据文件
│   ├── syllabus.txt             # 教学大纲（输入）
│   ├── questions.json           # 题库（持久化）
│   ├── task1_design_framework.txt  # Task1输出缓存
│   ├── task2_raw_json.txt       # Task2原始JSON
│   └── llm_full_output.txt      # LLM完整输出日志
│
├── question_generator.py          # 程序1：题目生成器（Sequential模式）
├── main.py                        # 程序2：考试系统（Hierarchical模式）
├── config.yaml                    # 业务配置（题目数量、模型参数等）
├── requirements.txt               # Python依赖
└── README.md                      # 项目说明
```

学习目标：这个实战项目教你什么

在深入代码之前，先明确本节的学习重点。这个实战项目不是简单的"照着敲代码"，而是要让你掌握将CrewAI应用到真实业务的核心能力。

核心学习目标

目标1：AI业务与传统业务的分界线

这是新手最容易犯的错误：以为用了AI框架，所有代码都要交给AI处理。实际上：

```
✓ AI负责的任务：
  • 创造性工作（生成题目、分析错题、个性化建议）
  • 需要理解语义的工作（分析大纲、理解学生错误模式）
  • 没有明确规则的决策

✗ 传统代码负责的任务：
  • 确定性逻辑（随机抽题、对比答案、计算分数）
  • 数据操作（读写JSON、数据库查询）
  • UI交互（显示界面、收集输入）
```

判断标准：如果一个任务可以用10行Python代码写出确定性逻辑，就不要用AI。AI的价值在于处理不确定性和创造性。

目标2：代码组织：函数式 vs 面向对象

CrewAI项目的典型代码结构：

```
agents/                    # 函数式：创建Agent的工厂函数
  question_expert.py       → create_question_expert(llm) -> Agent
  explanation_agent.py     → create_explanation_agent(llm) -> Agent

tasks/                     # 函数式：创建Task的工厂函数
  question_tasks.py        → create_design_task(agent, syllabus) -> Task
  explanation_tasks.py     → create_explanation_task(agent, result) -> Task

crews/                     # 面向对象：Crew作为业务流程的封装
  exercise_crew.py         → class ExerciseCrew (协调Agent和Task)

main.py                    # 面向对象：主程序封装业务逻辑
                           → class ExamSystem (完整业务流程)
```

设计原则：

设计原则

- Agent和Task用函数：它们是"配置"而非"状态机"，函数式更清晰

Agent和Task用函数

- Crew和主程序用类：它们需要管理状态（如LLM实例、配置），面向对象更合适

Crew和主程序用类

目标3：Crew、Agent、Task的协作模式

理解三者的职责边界和依赖关系：

```
┌──────────────────────────────────────────┐
  Crew (ExerciseCrew)
  • 职责：协调全局，决定执行模式
  • 持有：所有Agent实例、LLM配置
  • 方法：run_xxx() 封装不同业务流程
├──────────────────────────────────────────┤
  Agent (QuestionExpert, ExamAnalyzer...)
  • 职责：执行具体Task
  • 特性：角色定位、专业能力、背景故事
  • 依赖：LLM实例（由Crew传入）
├──────────────────────────────────────────┤
  Task (设计题目、生成题目...)
  • 职责：定义工作内容和输出要求
  • 关键：description（给Agent的Prompt）
  • 依赖：绑定的Agent、上游Task的输出
└──────────────────────────────────────────┘
```

关键问题：

关键问题

- Q: Agent和Task谁先创建？ A: Agent先创建，Task创建时绑定Agent
- Q: Task之间如何传递数据？ A: 通过context参数或Sequential模式自动传递
- Q: 一个Agent可以执行多个Task吗？ A: 可以，程序1中QuestionExpert执行了Task1和Task2

目标4：Sequential vs Hierarchical的选择依据

不要死记硬背模式特点，而要理解业务场景：

业务场景

场景特征	选择模式	本项目实例
任务有明确的先后顺序 （A完成后才能做B）	Sequential	程序1：设计框架→生成题目
多个任务需要并行执行 （可以同时进行）	Hierarchical	程序2：答案解析 + 学习建议
需要Manager动态分配任务	Hierarchical	程序2：Manager协调2个Agent
任务间没有强依赖关系	Hierarchical	-

经验法则：

经验法则

- 默认用Sequential（更简单、更容易调试）
- 只有在需要并行或动态协调时才用Hierarchical

学习路径建议

建议按以下顺序学习本节：

```
步骤1：先学程序1（question_generator.py）
  ├─ 理解最简单的Sequential流程
  ├─ 掌握Agent、Task、Crew的基本用法
  └─ 学会如何定义Prompt和输出格式

步骤2：再学程序2的阶段1-3（传统业务部分）
  ├─ 看清楚哪些逻辑不需要AI
  └─ 理解AI业务和传统业务如何衔接

步骤3：最后学程序2的阶段4（Hierarchical模式）
  ├─ 对比Sequential和Hierarchical的差异
  ├─ 理解Manager如何协调多个Agent
  └─ 学会HITL（Human-in-the-Loop）模式
```

带着问题去阅读

阅读代码时，思考这些问题：

- 程序1为什么要拆成2个Task？ 直接一个Task生成题目不行吗？

程序1为什么要拆成2个Task?

- 程序2的批改为什么不用AI？ 用AI批改不是更智能吗？

程序2的批改为什么不用AI?

- 为什么需要ExerciseCrew这个类？ 直接在main.py里创建Crew不行吗？
- Task的expected_output有什么用？ 不写会怎么样？

Task的expected_output有什么用？

- Hierarchical模式的Manager是怎么工作的？ 它会调用LLM吗？

Hierarchical模式的Manager是怎么工作的？

带着这些问题继续阅读，你会对架构设计有更深的理解。

下一节预告

第5节我们将深入剖析程序1（题目生成器），详细讲解Crew、Agent、Task三大组件如何协同工作，以及Sequential模式的执行流程。通过真实代码示例，你将掌握CrewAI框架的核心用法。